Building a Personalized Recommendation System

AUTHOR
Pascal Hermann Kouogang Tafo

INTRODUCTION

This assignment builds a personalized recommended system using survey data from "TV_Show_ratings.csv" that i manually created. The goal is to predict user preferences and recommend new TV shows, the users are likely to enjoy. To accomplish that goal, we will be implementing a **User-to-User Collaborative Filtering** algorithm using cosine similarity, generate top-N recommendations, and evaluate model performance.

APPROACH

The **User-to-User Collaborative Filtering** algorithm identifies users with similar tastes based on their past ratings and predicts ratings for items a user has not seen yet by **aggregating** the ratings of their "nearest neighbors". To build that personalized recommended system, we will be using the following approach:

- Load the **"TV_Show_ratings.csv"** from my public GitHub and convert it into a wide-format user-item matrix.
- Calculate the average rating for each user and subtract it from their ratings to account for individual rating biases.
- Use **Cosine Similarity** on the centered ratings to determine how similar users are to one another.
- For any unwatched show, calculate a weighted average of the ratings given by the most similar users.

Load the usefull library and the survey data

I will load the survey data that i created to build my personalized recommender system.

```
# Loading Library

library(tidyverse)
```

```
Warning: package 'tidyverse' was built under R version 4.5.2

Warning: package 'tibble' was built under R version 4.5.2

Warning: package 'tidyr' was built under R version 4.5.2

Warning: package 'readr' was built under R version 4.5.2
```

```
Warning: package 'purrr' was built under R version 4.5.2

Warning: package 'dplyr' was built under R version 4.5.2

Warning: package 'stringr' was built under R version 4.5.2

Warning: package 'forcats' was built under R version 4.5.2

Warning: package 'lubridate' was built under R version 4.5.2

── Attaching core tidyverse packages ──────────────────────── tidyverse 2.0.0 ──
✓ dplyr     1.2.0     ✓ readr     2.2.0
✓ forcats   1.0.1     ✓ stringr   1.6.0
✓ ggplot2   4.0.2     ✓ tibble    3.3.1
✓ lubridate 1.9.5     ✓ tidyr     1.3.2
✓ purrr     1.2.1
── Conflicts ────────────────────────────────────────── tidyverse_conflicts() ──
✕ dplyr::filter() masks stats::filter()
✕ dplyr::lag()    masks stats::lag()
ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
errors
```

```r
# Loading the survey data

TV_Show_ratings <- read_csv ("https://raw.githubusercontent.com/Pascaltafo2025/Assignment-11-Pers
```

```
Rows: 5 Columns: 7
── Column specification ────────────────────────────────────────────────────────
Delimiter: ","
chr (1): user_id
dbl (6): stranger_things, breaking_bad, the_crown, the_witcher, money_heist,...

ℹ Use `spec()` to retrieve the full column specification for this data.
ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

ALGORITHM CHOICE & IMPLEMENTATION

Step 1: Create User-Item Matrix

```r
ratings_matrix <- TV_Show_ratings

ratings_matrix <- as.data.frame(ratings_matrix)
row.names(ratings_matrix) <- ratings_matrix$user_id
ratings_matrix$user_id <- NULL
```

```r
ratings_matrix
```

```
      stranger_things breaking_bad the_crown the_witcher money_heist squid_game
Alain               5            4         5          NA           4         NA
John                4            5        NA           3           4          5
Dany                5           NA         4           4          NA          5
Jesse              NA            5         4          NA           3          4
Sarah               4            5        NA           3           4         NA
```

Step 2: Compute User Similarity (Cosine Similarity)

```r
cosine_sim <- function(x, y) {
  round(
    sum(x * y, na.rm = TRUE) /
      (sqrt(sum(x^2, na.rm = TRUE)) * sqrt(sum(y^2, na.rm = TRUE))),
    2
  )
}

user_similarity <- matrix(
  NA,
  nrow = nrow(ratings_matrix),
  ncol = nrow(ratings_matrix)
)

for (i in 1:nrow(ratings_matrix)) {
  for (j in 1:nrow(ratings_matrix)) {
    user_similarity[i, j] <- cosine_sim(
      as.numeric(ratings_matrix[i, ]),
      as.numeric(ratings_matrix[j, ])
    )
  }
}

rownames(user_similarity) <- rownames(ratings_matrix)
colnames(user_similarity) <- rownames(ratings_matrix)

user_similarity
```

```
      Alain John Dany Jesse Sarah
Alain  1.00 0.65 0.55  0.71  0.76
John   0.65 1.00 0.66  0.74  0.85
Dany   0.55 0.66 1.00  0.49  0.43
Jesse  0.71 0.74 0.49  1.00  0.56
Sarah  0.76 0.85 0.43  0.56  1.00
```

Step 3: Predict Missing Ratings and Generate Predictions

```r
# Predict Missing Ratings function

predict_rating <- function(user, item) {
  sims <- user_similarity[user, ]
  ratings <- ratings_matrix[, item]

  valid <- !is.na(ratings)

  sum(sims[valid] * ratings[valid]) / sum(abs(sims[valid]))
}

# Generate Predictions

predictions <- ratings_matrix

for(u in 1:nrow(ratings_matrix)){
  for(i in 1:ncol(ratings_matrix)){
    if(is.na(ratings_matrix[u, i])){
      predictions[u, i] <- round(predict_rating(u, i),2)
    }
  }
}

predictions
```

```
      stranger_things breaking_bad the_crown the_witcher money_heist squid_game
Alain            5.00         4.00      5.00        3.28        4.00       4.63
John             4.00         5.00      4.32        3.00        4.00       5.00
Dany             5.00         4.74      4.00        4.00        3.77       5.00
Jesse            4.48         5.00      4.00        3.27        3.00       4.00
Sarah            4.00         5.00      4.43        3.00        4.00       4.70
```

Top Recommendation per User

Here, we will display recommended TV shows for a specific user and compute a practical example.

```r
get_top_N <- function(user_index, n=2) {
  user_ratings <- ratings_matrix[user_index, ]
  user_preds <- predictions[user_index, ]

  unseen <- is.na(user_ratings)

  top_items <- sort(user_preds[unseen], decreasing = TRUE)

  head(top_items, n)
}

# Example for user 1
```

```
get_top_N(1, 2)
```

```
[1] 4.63 3.28
```

Interpretation:

The recommender system outputs a ranked list of the top-N TV shows for each user that they have not yet rated, based on predicted ratings derived from similar users. Indeed, Alain who is the first user was recommended squid_game first and then the_witcher.

Model Evaluation

```
# Convert from wide format to long format
ratings_long <- TV_Show_ratings %>%
  pivot_longer(
    cols = -user_id,
    names_to = "show",
    values_to = "rating"
  ) %>%
  filter(!is.na(rating))


set.seed(123)

ratings_split <- ratings_long %>%
  group_by(user_id) %>%
  mutate(test = sample(c(TRUE, FALSE), n(), replace = TRUE, prob = c(0.2, 0.8)))

train <- ratings_split %>% filter(test == FALSE)
test  <- ratings_split %>% filter(test == TRUE)

#RMSE Function

rmse <- function(actual, predicted) {
  sqrt(mean((actual - predicted)^2, na.rm = TRUE))
}
# Evaluate Model

test$predicted <- mapply(function(u, i) {
  predict_rating(u, i)
}, test$user_id, test$show)

# Calculation of Root Mean Squared Error (RMSE)
```

```r
rmse_value <- rmse(test$rating, test$predicted)
rmse_value
```

```
[1] 0.346216
```

CONCLUSION

This assignment successfully developed a personalized recommender system using User-to-User Collaborative Filtering to predict user preferences and generate top-N TV show recommendations. By applying cosine similarity, predicting missing ratings, and evaluating performance using RMSE, the model demonstrated how recommendation systems can provide meaningful personalized suggestions from survey data.